

EXAMEN DE PROGRAMACIÓN – 1º GRADO SUPERIOR

Examen 8 · Colecciones y genéricos

La interfaz List: ArrayList y LinkedList. Iterator

Conjuntos: HashSet, LinkedHashSet y TreeSet. Orden e igualdad

Mapas: HashMap, LinkedHashMap y TreeMap

Comparable vs Comparator. Ordenación de colecciones

Genéricos: clases y métodos genéricos. Autoboxing en colecciones

Asignatura	Programación (PRG)
Curso	1º Grado Superior — Desarrollo de Aplicaciones Web (DAW)
Duración	2 horas
Puntuación total	16 puntos
Convocatoria	Evaluación 2
Valoración	Colecciones y genéricos

Parte 1: Opción múltiple y Verdadero/Falso

4 puntos — 0,5 ca

¿Qué implementación de List es más eficiente para insertar y eliminar elementos en posiciones intermedias?

- a) ArrayList
- b) LinkedList
- c) TreeSet
- d) HashMap

RESPUESTA / SOLUCIÓN

b) LinkedList — La inserción/eliminación en medio solo requiere reenlazar nodos; en ArrayList hay que desplazar todos los elementos posteriores.

¿Qué imprime el siguiente código?

```
TreeSet<Integer> ts = new TreeSet<>();  
ts.add(5); ts.add(1); ts.add(5); ts.add(3);  
System.out.println(ts.size() + " " + ts);
```

- a) 4 [1, 3, 5, 5]
- b) 3 [1, 3, 5]
- c) 3 [5, 3, 1]
- d) 4 [5, 3, 1, 5]

RESPUESTA / SOLUCIÓN

b) 3 [1, 3, 5] — TreeSet no admite duplicados (el segundo 5 se descarta) y mantiene el orden natural ascendente.

Dado el código, ¿qué se imprime?

```
Map<String, Integer> m = new HashMap<>();  
m.put("a", 1);  
System.out.println(m.put("a", 2));
```

- a) null
- b) 1
- c) 2
- d) a

RESPUESTA / SOLUCIÓN

b) 1 — put() devuelve el valor anterior asociado a la clave (o null si no existía) y después lo sustituye por 2.

HashSet permite almacenar elementos duplicados.

RESPUESTA / SOLUCIÓN

Falso — HashSet rechaza duplicados: si se añade un elemento igual (según equals/hashCode) a uno existente, add() devuelve false y no lo inserta.

Al recorrer un ArrayList con for-each y llamar a remove() dentro del bucle se lanza...

- a) NullPointerException
- b) ConcurrentModificationException

- c) `IndexOutOfBoundsException`
- d) No lanza ninguna excepción

RESPUESTA / SOLUCIÓN

b) `ConcurrentModificationException` — El iterador del `for-each` detecta que la lista se modificó y falla; hay que usar `Iterator` y su método `remove()`.

Para que una clase tenga un orden natural y pueda usarse en `TreeSet` o `Collections.sort()` debe implementar...

- a) `Comparator` con `compare()`
- b) `Comparable` con `compareTo()`
- c) `Iterator` con `hasNext()`
- d) `Serializable`

RESPUESTA / SOLUCIÓN

b) `Comparable` con `compareTo()` — El orden natural se define implementando `Comparable`; `Comparator` permite definir órdenes alternativos sin tocar la clase.

Dada la clase genérica `class Caja<T> { ... }`, ¿qué tipo concreto tiene `T` en la declaración `Caja<String> c = new Caja<>();`?

- a) `String`
- b) `Object`
- c) `T`
- d) Error de compilación

RESPUESTA / SOLUCIÓN

a) `String` — El parámetro de tipo se instancia con `String`; el operador diamante `<>` infiere el tipo en el constructor.

`TreeMap` mantiene sus claves ordenadas según el orden natural de la clave o el `Comparator` proporcionado.

RESPUESTA / SOLUCIÓN

Verdadero — `TreeMap` ordena las claves (orden natural o `Comparator`); `HashMap` no garantiza ningún orden y `LinkedHashMap` mantiene el orden de inserción.

Parte 2: Análisis y depuración

3 puntos

El siguiente programa tiene 2 errores. Encuéntralos, explica por qué fallan y propón la corrección. (1,5 pts)

```
import java.util.*;
public class BorrarPares {
    public static void main(String[] args) {
        List<Integer> numeros = new ArrayList<>(List.of(1, 2, 3, 4, 5, 6));
        for (Integer n : numeros) {
            if (n % 2 == 0) numeros.remove(n);
        }
        Integer x = numeros.get(0);
        Integer y = numeros.get(1);
        System.out.println(x == y);
    }
}
```

RESPUESTA / SOLUCIÓN

1. Modificar la lista con remove() durante un for-each lanza ConcurrentModificationException.

Corrección: recorrer con Iterator y usar it.remove():

```
Iterator<Integer> it = numeros.iterator();
```

```
while (it.hasNext()) { if (it.next() % 2 == 0) it.remove(); }
```

2. x == y compara referencias de Integer, no valores: si ambos son iguales pero están fuera de la caché (-128..127) daría false. Corrección: x.equals(y) o x.intValue() == y.intValue().

Analiza el siguiente código y determina qué orden imprime el programa (traza manual). (1,5 pts)

```
List<Alumno> lista = new ArrayList<>>();
lista.add(new Alumno("Ana", 7.5));
lista.add(new Alumno("Luis", 5.0));
lista.add(new Alumno("Eva", 9.2));
lista.add(new Alumno("Juan", 6.3));
lista.sort((a1, a2) -> Double.compare(a2.nota, a1.nota));
for (Alumno a : lista) System.out.println(a.nombre);
```

RESPUESTA / SOLUCIÓN

El comparador usa Double.compare(a2.nota, a1.nota): al invertir los operandos, el orden queda DESCENDENTE por nota.

Salida: Eva, Ana, Juan, Luis.

Parte 3: Ejercicios de programación

8 puntos

Escribe un programa que cuente cuántas veces aparece cada palabra en una frase y muestre el resultado con un `HashMap<String, Integer>`. (2 ptos)

RESPUESTA / SOLUCIÓN

```
import java.util.*;

public class Frecuencias {

    public static void main(String[] args) {

        String frase = "el sol brilla y el sol calienta";

        String[] palabras = frase.split(" ");

        Map<String, Integer> contador = new HashMap<>();

        for (String p : palabras) {

            contador.put(p, contador.getOrDefault(p, 0) + 1);

        }

        for (String p : contador.keySet()) {

            System.out.println(p + " -> " + contador.get(p));

        }

    }

}
```

Salida: el -> 2, sol -> 2, brilla -> 1, y -> 1, calienta -> 1 (el orden de HashMap puede variar).

Implementa una clase genérica `Par<K, V>` que almacene una clave y un valor (con `getClave()`, `getValor()` y `setValor()`). Úsala en un main con `Par<String, Integer>` y `Par<String, String>`. (2 ptos)

RESPUESTA / SOLUCIÓN

```
public class Par<K, V> {

    private K clave;

    private V valor;

    public Par(K clave, V valor) { this.clave = clave; this.valor = valor; }

    public K getClave() { return clave; }

    public V getValor() { return valor; }

    public void setValor(V valor) { this.valor = valor; }

    public static void main(String[] args) {

        Par<String, Integer> p = new Par<>("edad", 25);

        System.out.println(p.getClave() + " = " + p.getValor());

        p.setValor(26);

        System.out.println(p.getClave() + " = " + p.getValor());

        Par<String, String> q = new Par<>("ciudad", "Madrid");

        System.out.println(q.getClave() + " = " + q.getValor());

    }

}
```

```
}
```

Crea una clase Alumno (nombre, nota) y un programa que guarde varios alumnos en un ArrayList, los ordene por nota descendente con Comparator y muestre también la nota media. (2 ptos)

RESPUESTA / SOLUCIÓN

```
import java.util.*;

public class OrdenarAlumnos {

    static class Alumno {

        String nombre; double nota;

        Alumno(String nombre, double nota) { this.nombre = nombre; this.nota = nota; }

        public String toString() { return nombre + " (" + nota + ")"; }

    }

    public static void main(String[] args) {

        List<Alumno> lista = new ArrayList<>();

        lista.add(new Alumno("Ana", 7.5));

        lista.add(new Alumno("Luis", 5.0));

        lista.add(new Alumno("Eva", 9.2));

        lista.add(new Alumno("Juan", 6.3));

        lista.sort((a1, a2) -> Double.compare(a2.nota, a1.nota));

        double suma = 0;

        for (Alumno a : lista) suma += a.nota;

        System.out.println("Media: " + (suma / lista.size()));

        for (Alumno a : lista) System.out.println(a);

    }

}
```

Salida: Media: 7.0; Eva (9.2), Ana (7.5), Juan (6.3), Luis (5.0).

Usa un TreeSet con un Comparator que ordene las palabras primero por longitud y después alfabéticamente. Inserta {"sol", "mar", "cielo", "sol", "luna"} y muestra el tamaño y el contenido final. (2 ptos)

RESPUESTA / SOLUCIÓN

```
import java.util.*;

public class OrdenarLongitud {

    public static void main(String[] args) {

        Comparator<String> porLongitud = new Comparator<>() {

            public int compare(String a, String b) {

                if (a.length() != b.length()) return a.length() - b.length();

                return a.compareTo(b);

            }

        };

    }

};
```

```
    TreeSet<String> set = new TreeSet<>(porLongitud);  
    set.add("sol"); set.add("mar"); set.add("cielo");  
    set.add("sol"); set.add("luna");  
    System.out.println(set.size());  
    System.out.println(set);  
}  
}
```

Salida: tamaño 4 ("sol" duplicado se descarta) y contenido [mar, sol, luna, cielo].

Parte 4: Pregunta teórica

1 punto

Explica las diferencias entre List, Set y Map. ¿Qué implementación elegirías para: (a) acceso frecuente por índice, (b) evitar duplicados conservando el orden de inserción, (c) buscar valores por clave rápidamente? Razona la respuesta.

RESPUESTA / SOLUCIÓN

List: secuencia ordenada que admite duplicados y acceso por índice. Set: colección sin duplicados (igualdad por equals/hashCode). Map: pares clave-valor sin claves repetidas.

(a) ArrayList: acceso por índice $O(1)$. (b) LinkedHashSet: sin duplicados y mantiene orden de inserción (HashSet no garantiza orden; TreeSet ordena, no conserva inserción). (c) HashMap: búsqueda por clave $O(1)$ media gracias al hashCode.

Plantilla de respuestas

(Páginas en blanco)